

Events

Willkommen!

Sämtliche Notizen sind nur zur Unterstützung für den Trainer gedacht.



Events

- Statischer Punkt, an den ein Delegate angehängt werden kann
 - Mehrere Delegates möglich (aber unwahrscheinlich)
- Methoden im Code können diese Events ausführen
- Wenn das Event ausgeführt wird, wird die angehängte Methode ausgeführt
- Zweigeteilte Programmierung
 - Entwicklerseite
 - Anwenderseite



Entwicklerseite/Anwenderseite

Entwicklerseite

- Legt das Event selbst an
- Feuert das Event im Code
- Gibt Daten an die Empfänger-Methode weiter

Anwenderseite

- Hängt ein Delegate an den Eventpunkt an
- Legt fest, was das Event machen soll
- Empfängt die Daten, welche bei Eventausführung gesendet werden



EventHandler

- ein Event ist eine Delegate-Variable, an der sich beliebig viele Methoden (EventListener) an- und abmelden können
- Delegate-Typ ist i.d.R. EventHandler
 - Erwartete Übergabeparameter: `object`, `EventArgs`
- Kein „Überschreiben“ durch Neuzuweisung möglich

```
event EventHandler onClick;  
  
onClick += delegate (object sender, EventArgs eventArgs)  
{  
    Console.WriteLine("EventBehandlung");  
};  
  
onClick(this, EventArgs.Empty);
```

event => keine direkt zuweisung über = möglich

EventArgs hat standardmäßig keine Eigenschaften zur Übergabe
man muss eigene Klasse erstellen, welche von EventArgs erbt

EventHandler bekommt die ArgsKlasse als generischen Typ übergeben
EventHandler<MeineEventArgs> onClick = ...



EventArgs

- Über den EventArgs-Parameter bei Events werden Daten an die angehängte Methode weitergegeben
- Daten:
 - Viele verschiedene C#-interne EventArgs Varianten
 - Eigene EventArgs
 - Eigene Klasse mit Oberklasse „EventArgs“
 - Beliebige Properties, um Daten zu bewegen
- Keine Daten: `EventArgs.Empty`

```
public class CustomEventArgs : EventArgs
{
    public readonly string Status;

    1 reference | 0 changes | 0 authors, 0 changes
    public CustomEventArgs(string status)
    {
        Status = status;
    }
}
```



Event Accessoren

- Events können auf Hinzufügen und/oder entfernen von Methoden reagieren
- Definition von einem private Event und einem public Event mit add- und remove Accessoren

```
private event EventHandler accessorEvent;  
  
public event EventHandler AccessorEvent  
{  
    add => accessorEvent += value;  
    remove => accessorEvent -= value;  
}
```